

Aula 15 (Prática): PageRank, HITS e GNNs na Mão

Prof. Eliezer de Souza da Silva

MDCC – Universidade Federal do Ceará

1 Objetivos de Aprendizagem

Nesta aula prática, vamos ligar a teoria de difusão em grafos e message passing com implementações concretas em PyTorch. Ao final, você deve ser capaz de:

- Implementar e executar PageRank e HITS em um grafo pequeno, vendo-os como **camadas lineares fixas**.
- Implementar três camadas de GNN:
 - GCN,
 - GraphSAGE (com `forward` a ser completado),
 - GAT (código completo para referência).
- Implementar uma GNN simples (**SimpleGNNLayer**) que faz agregação + update via MLP.
- Explorar, na prática, o **loop de message passing**: aplicar várias vezes a mesma camada.

2 Parte 0: Setup no Colab

Crie um novo notebook no Google Colab e execute:

```
!pip install networkx --quiet

import torch
import torch.nn as nn
import torch.nn.functional as F
import networkx as nx
import random, math

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

3 Parte 1: Representando Grafos

Vamos começar com um grafo dirigido simples de 4 nós:

```
A = torch.tensor([
    [0., 1., 1., 0.],
    [0., 0., 1., 0.],
    [0., 0., 0., 1.],
    [1., 0., 0., 0.]
])
```

```
], dtype=torch.float32, device=device)

N = A.shape[0]
X_small = torch.eye(N, device=device) # features identidade
```

Exercício 1 (Aquecimento)

- (a) Calcule o **grau de saída** de cada nó (soma por linha de A).
- (b) Crie a matriz de transição P normalizando as linhas de A :

$$P_{ij} = \frac{A_{ij}}{\sum_j A_{ij}}.$$

- (c) Verifique no código que cada linha de P soma 1.

4 Parte 2: PageRank como Módulo Linear (com TODO)

Relembrando a fórmula do PageRank com fator de amortecimento α :

$$\mathbf{r}^{(k+1)} = \alpha P^\top \mathbf{r}^{(k)} + (1 - \alpha) \frac{1}{N} \mathbf{1}.$$

Vamos construir uma camada PageRankLayer que implementa exatamente isso.

Código-base com lacunas

```
def row_normalize(A):
    deg = A.sum(dim=-1, keepdim=True)
    deg = torch.where(deg == 0, torch.ones_like(deg), deg)
    return A / deg

P = row_normalize(A)
P_t = P.t()

class PageRankLayer(nn.Module):
    def __init__(self, P_t):
        super().__init__()
        N = P_t.shape[0]
        self.linear = nn.Linear(N, N, bias=False)
        # TODO 1: carregar P_t nos pesos da camada linear
        with torch.no_grad():
            # substitua "..." pela linha correta
            # self.linear.weight.copy_( ...)
            ...

        # TODO 2: desativar gradientes (PageRank nao e treinado)
        for p in self.linear.parameters():
            # p.requires_grad = ???
            ...

    def forward(self, r, alpha=0.85):
        # r: vetor de PageRank atual [N]
        # alpha: fator de amortecimento

        N = r.shape[0]
```

```

# TODO 3: calcule a parte de caminhada aleatoria com a camada linear
# walk = ...
...

# TODO 4: calcule o vetor de teleporte uniforme
# teleport = ...
...

# TODO 5: combine walk e teleport
# return ...
...

```

Exercício 2

- (a) Preencha os cinco TODOs acima.
- (b) Inicialize o vetor de PageRank:

```
r = torch.ones(N, device=device) / N
```

- (c) Aplique PageRankLayer por 50 iterações, guardando os vetores em uma lista.
- (d) Plote a evolução do score de um nó específico ao longo das iterações.

5 Parte 3: HITS com Módulos Lineares (com TODO)

No HITS, temos dois vetores:

$$\mathbf{a}^{(k+1)} = A^T \mathbf{h}^{(k)}, \quad \mathbf{h}^{(k+1)} = A \mathbf{a}^{(k+1)}.$$

Depois, normalizamos cada um pela sua norma.

Código-base com lacunas

```

class HITSLayer(nn.Module):
    def __init__(self, A):
        super().__init__()
        N = A.shape[0]
        self.A = nn.Linear(N, N, bias=False)
        self.A_t = nn.Linear(N, N, bias=False)
        with torch.no_grad():
            self.A.weight.copy_(A)
            self.A_t.weight.copy_(A.t())
        # TODO 1: desativar gradientes (HITS nao eh treinado)
        for p in self.parameters():
            # p.requires_grad = ???
            ...

    def forward(self, a, h):
        # a: vetor de authority atual [N]
        # h: vetor de hub atual [N]

        # TODO 2: atualizar a usando A^T e h
        # a_next = ...
        ...

```

```

# TODO 3: normalizar a_next pela norma L2
# a_next = a_next / (a_next.norm() + 1e-8)
...

# TODO 4: atualizar h usando A e a_next
# h_next = ...
...

# TODO 5: normalizar h_next
# h_next = ...
...

return a_next, h_next

```

Exercício 3

- Preencha os TODOs da classe HITSLayer.
- Inicialize:

```

a = torch.ones(N, device=device)
h = torch.ones(N, device=device)
a = a / a.norm()
h = h / h.norm()

```

- Execute 50 iterações de HITS.
- Imprima os vetores finais e identifique:
 - os nós com maior authority;
 - os nós com maior hub.

6 Parte 4: Camadas de GNN

Agora vamos criar três camadas de GNN. Todas recebem:

$$X \in \mathbb{R}^{N \times F}, \quad A \in \mathbb{R}^{N \times N}.$$

6.1 GCNLayer (completo para referência)

```

def gcn_norm(A):
    I = torch.eye(A.shape[0], device=A.device)
    A_tilde = A + I
    deg = A_tilde.sum(dim=-1)
    D_inv_sqrt = torch.diag(torch.pow(deg, -0.5))
    return D_inv_sqrt @ A_tilde @ D_inv_sqrt

class GCNLayer(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.linear = nn.Linear(in_dim, out_dim)

    def forward(self, X, A_hat):
        H = A_hat @ X
        return F.relu(self.linear(H))

```

6.2 GraphSAGELayer (com TODO no forward)

A fórmula que queremos implementar é:

$$m_i^{(k)} = \text{MEAN}(\{h_j^{(k)} : j \in \mathcal{N}(i)\}), \quad h_i^{(k+1)} = \sigma(W[h_i^{(k)} \parallel m_i^{(k)}]).$$

```
class GraphSAGELayer(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.linear = nn.Linear(2 * in_dim, out_dim)

    def forward(self, X, A):
        """
        =====X: [N, F]
        =====A: [N, N] matriz de adjacencia (sem normalizar)
        =====Objetivo:
        =====1) calcular a media dos vizinhos de cada no;
        =====2) concatenar X com a media de vizinhos;
        =====3) aplicar a camada linear + ReLU.
        =====
        # TODO 1: calcular graus (soma por linha)
        # deg = ...
        ...

        # TODO 2: evitar divisao por zero (clamp(min=1.))
        # deg = deg.clamp(min=1.)
        ...

        # TODO 3: calcular a media dos vizinhos: neigh_mean = (A @ X) / deg
        # neigh_mean = ...
        ...

        # TODO 4: concatenar X e neigh_mean ao longo da dimensao de features
        # H_cat = torch.cat([...], dim=-1)
        ...

        # TODO 5: aplicar linear + ReLU
        # return F.relu(self.linear(H_cat))
        ...
```

6.3 GATLayer (completo para referência)

```
class GATLayer(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.W = nn.Linear(in_dim, out_dim, bias=False)
        self.a = nn.Linear(2 * out_dim, 1, bias=False)
        self.leaky_relu = nn.LeakyReLU(0.2)

    def forward(self, X, A):
        H = self.W(X) # [N, F']
        N = H.size(0)
        H_i = H.unsqueeze(1).repeat(1, N, 1)
        H_j = H.unsqueeze(0).repeat(N, 1, 1)
        e_ij = self.leaky_relu(self.a(torch.cat([H_i, H_j], dim=-1))).squeeze(-1)
        e_ij = e_ij.masked_fill(A == 0, float('-inf'))
```

```

alpha = torch.softmax(e_ij, dim=-1)
H_prime = alpha @ H
return F.relu(H_prime)

```

Exercício 4

- (a) Complete o forward da GraphSAGELayer.
- (b) Teste a camada com um grafo pequeno (por exemplo, o grafo de 4 nós acima).
- (c) Compare a saída da GraphSAGE com a da GCN quando A é o mesmo.

7 Parte 5: GNN Simples (SimpleGNNLayer) + Loop de Message Passing

Agora vamos implementar uma GNN mínima baseada na fórmula:

$$m_i^{(k)} = \text{MEAN}(\{h_j^{(k)} : j \in \mathcal{N}(i)\}), \quad h_i^{(k+1)} = \sigma(\text{MLP}([h_i^{(k)} \parallel m_i^{(k)}])).$$

Código-base com TODO

```

class SimpleGNNLayer(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        super().__init__()
        # MLP simples: [h_i || m_i] -> out_dim
        self.mlp = nn.Sequential(
            nn.Linear(2 * in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim)
        )

    def forward(self, X, A):
        """
        =====X: [N, F] (representacoes_atuais_dos_nos)
        =====A: [N, N] (adjacencia)
        =====
        # TODO 1: calcular deg = soma por linha
        # deg = ...
        ...

        # TODO 2: evitar divisao por zero
        # deg = deg.clamp(min=1.)
        ...

        # TODO 3: calcular m_i = media dos vizinhos
        # m = ...
        ...

        # TODO 4: concatenar X e m
        # H_cat = torch.cat([...], dim=-1)
        ...

        # TODO 5: aplicar o MLP
        # return self.mlp(H_cat)
        ...

```

Exercício 5 (Loop de Message Passing)

- (a) Complete o forward da SimpleGNNLayer.
- (b) Escolha um grafo pequeno (por exemplo, Karate Club ou o grafo dos 4 nós).
- (c) Inicialize $H^{(0)} = X$ (por exemplo, identidade).
- (d) Aplique a mesma camada SimpleGNNLayer várias vezes, **sem treinar** os pesos:

```
layer = SimpleGNNLayer(in_dim=..., hidden_dim=16, out_dim=...).to(
    device)
H = X.clone()

for k in range(5):
    H = layer(H, A)
    print(f"Iteracao_{k+1}, _norma_media:_{H.norm(dim=-1).mean().item()
        :.4f}")
```

- (e) Observe:
 - Os vetores dos nós tendem a se parecer mais entre si?
 - Nós na mesma comunidade ficam mais próximos?
- (f) Relacione com a ideia de difusão vista na Aula 14 (Laplaciano).

8 Parte 6: Karate Club (Classificação de Nós)

Código-base (pode usar GCN, GraphSAGE ou GAT)

```
G_karate = nx.karate_club_graph()
N_k = G_karate.number_of_nodes()
A_k_np = nx.to_numpy_array(G_karate)
A_k = torch.tensor(A_k_np, dtype=torch.float32, device=device)
X_k = torch.eye(N_k, device=device)

labels = []
for i in range(N_k):
    club = G_karate.nodes[i]['club']
    labels.append(0 if club == 'Mr._Hi' else 1)
y_k = torch.tensor(labels, dtype=torch.long, device=device)

idx_all = list(range(N_k))
random.seed(0)
random.shuffle(idx_all)
split = N_k // 2
idx_train = torch.tensor(idx_all[:split], dtype=torch.long, device=device)
idx_test = torch.tensor(idx_all[split:], dtype=torch.long, device=device)

A_k_hat = gcn_norm(A_k)
```

Você pode montar modelos de 2 camadas (por exemplo, GCNLayer ou GraphSAGELayer) e treinar para classificar os nós.

Exercício 6

- (a) Treine um modelo GCN de 2 camadas para prever as comunidades do Karate Club.
- (b) Repita usando GraphSAGE (usando sua implementação do `forward`).
- (c) Compare as acurácias de treino e teste.

9 Parte 7: Mini Sistema de Recomendação Usuário–Item

Use o código do professor (ou adaptação) para construir um grafo usuário–item, treinar uma GraphSAGE ou SimpleGNN para aprender embeddings e gerar recomendações.

Exercício 7

- (a) Monte um grafo bipartido simples com 3 usuários e 4 itens.
- (b) Treine uma GNN (por exemplo, GraphSAGE) para aprender embeddings.
- (c) Para cada usuário, recomende 2 itens que ainda não têm aresta, usando produto interno entre embeddings.

10 Desafios Extras

- Experimente diferentes profundidades (1, 2, 3 camadas) para a SimpleGNN e observe over-smoothing.
- Adicione dropout e compare desempenho.
- Substitua a média de vizinhos por soma em `GraphSAGELayer` e observe o efeito.